

ログからのインシデント検出と 自動改修の運用確立

複数サーバに分散したログを、AIエージェントで1件のインシデントに束ねる

研修環境 `projects/onprem_log_training` のデモの流れに沿って

対象: マネジメント・意思決定層 / 想定所要 20分

エグゼクティブサマリ

1. 問題 — 障害調査の大半は「原因を考える時間」ではなく、**複数サーバのログを human が突き合わせる時間**に消えている。

2. 打ち手 — 全リクエストに共通の相関キー(**TrackID**)を通し、収集・突き合わせをAIエージェントに任せる。人は判断に集中する。

3. 進め方 — ツールを買うのではなく、**運用として定着させる**。そのために、担当者自身がAIエージェントで収集ツールを組み立てる研修環境を用意した。

検知→収集の自動化を今期の主対象とし、**改修PR自動発行までの延長線は実証済み**(後述)。

現状: 障害調査は「ログ探し」に消えている

典型的な調査フロー(すべて人手):

#	作業	実態
1	「500エラーが出た」と連絡が入る	どのサーバの問題か、この時点では不明
2	アプリサーバのログを見る	エラーは見えるが、原因は上流にある
3	機能サーバにSSHして <code>grep</code>	接続情報・ログの場所を知っている人しか動けない
4	2つのログを時刻で目視突き合わせ	秒単位で並ぶログから該当処理を推測
5	チケットに転記	コピペ作業

構造的な問題は 3 と 4 — 属人化(担当者しか調べられない)と 推測による突き合わせ(時刻に近いログを「たぶん同じ処理」と見なす)。

なぜ「時刻で突き合わせ」が破綻するか

秒間リクエストが増えると

同じ 100ms の中に無関係なリクエストが何本も並ぶ。

時刻の近さは**同一処理の証明にならない**。

サーバをまたぐと

ログの時刻はサーバごとの時計に依存する。
処理順と記録順が逆転して見えることもある。

必要なのは推測ではなく、**処理そのものに付いた一意の識別子**。

```
app.log      (アプリサーバ)
09:00:00.123 ERROR /api/robots/RBT-DOG-02
09:00:00.124 INFO  /api/cart
09:00:00.125 ERROR /api/orders

service.log (機能サーバ)
09:00:00.089 ERROR /internal/inventory/...
09:00:00.091 INFO  /internal/billing/calc
09:00:00.093 ERROR /internal/billing/calc
```

どの ERROR とどの ERROR が同じ処理か、時刻だけでは決められない。

打ち手: TrackID による相関

アプリサーバが受けたリクエスト1件ごとに **7文字の TrackID** を発行し、
下流の機能サーバへ HTTP ヘッダ `X-Track-Id` で引き渡す。

ブラウザ操作

↓

client (アプリサーバ層) TrackID:ABC1234 を発行 → app.log に記録

↓

X-Track-Id: ABC1234 を付けて内部API呼び出し

↓

server (機能サーバ層) 受け取った ABC1234 を service.log に記録

結果として、**2つのログファイルに同じ TrackID が残る。**

突き合わせが「推測」から **TrackID:ABC1234 の完全一致検索** に変わる。

正規表現 `TrackID:([A-Z0-9]{7})` で機械的に抽出できる = **自動化できる。**

打ち手はツール導入ではなく運用定着

ツールを配るだけでは

- 環境が変わると誰も直せない
- 「なぜそう動くか」を知る人がいない
- 結果、また属人化する

担当者が自分で作れば

- 自分の環境に合わせて直せる
- AIエージェントへの指示の型が身につく
- **他の運用課題にも応用が効く**

そのための環境が **projects/onprem_log_training**。

この環境には **ログ収集ツールをあえて同梱していない** — 受講者がAIエージェントと組み立てる対象。

用意してあるのは「関連できるログを吐く、リアルな障害環境」と、
AIエージェントに読ませるための接続情報（**AGENT_SSH_GUIDE.md**）。

研修環境の構成

2つのサーバ役を Docker コンテナで再現。あえて非対称に作っている。

役割	担当	ログの置き場所	SSH
client アプリサーバ層	画面・API を提供 (ポート 3002)	client/logs/app.log ホストにbind mount	非搭載
server 機能サーバ層	在庫確認・注文金額計算 (ポート 4002)	/app/logs/service.log コンテナ内のみ	搭載 (ポート 5101)

非対称にした理由 — 片方(client)は簡単に読め、片方(server)は**SSH経由でしか読めない**。
「手元で見えるログだけで満足するツール」では解けない構造を、意図的に作っている。

起動は `cd docker && ./setup-containers.sh start` の1コマンド。ブラウザで `http://localhost:3002/` 。

題材: 架空のEC「RoboMart」と仕込まれた不具合

機能サーバ側の業務ロジックに、実務でよく見る型の不具合を2つ常設。

不具合ID	再現操作	技術的な原因	業務影響
PRODUCT_STOCK_ZERO_NPE	在庫切れ商品 「WalkyDog Mk2」 の詳細ページを開く	在庫0の分岐で存在しない フィールドを <code>.map()</code> し TypeError	在庫切れページが 500 → 離脱率上昇
ORDER_TOTAL_UNDEFINED_TAX	決済方法 「請求書払い」 で注文確定	税率が未定義になり <code>total.toFixed()</code> で TypeError	法人向け請求書決済が 全減、機会損失甚大

いずれも `server/bug-config.json` の `enabled` で個別にON/OFF。
再現性が担保されているため、研修・リハーサルで毎回同じ障害を起こせる。

デモの流れ

バグ発火から、1件のインシデントに束ねるまで

デモ ① バグを踏む (0～10秒)

操作: ブラウザで在庫切れ商品「WalkyDog Mk2」の詳細ページを開く。

画面: 500エラー。レスポンスに `track_id` が含まれる。

この瞬間、2つのログに同じTrackIDが書かれる:

```
# client/logs/app.log (ホストから直接読める)
2026-07-25T09:00:00.123Z ERROR TrackID:ABC1234 [/api/robots/RBT-DOG-02] method=GET status=500 \
  err=TypeError: Cannot read properties of undefined (reading 'map')

# /app/logs/service.log (serverコンテナ内、SSH経由でのみ読める)
2026-07-25T09:00:00.089Z ERROR TrackID:ABC1234 [/internal/inventory/RBT-DOG-02] method=GET status=500 \
  err=TypeError: Cannot read properties of undefined (reading 'map')
```

両ログは**同一フォーマット**:

<ISO8601> <LEVEL> TrackID:<7文字> [<パス>] method=<M> key=value ...

デモ ② 検知し、TrackIDを取り出す

AIエージェントに作らせたツールが `app.log` を監視し、`ERROR` 行を見つける。

検知した行 → 正規表現 `TrackID:([A-Z0-9]{7})` で抽出 → `ABC1234`

ここまでは簡単

`app.log` はホストのローカルファイル。
読むだけなら誰でもできる。

だが情報が足りない

アプリサーバ側のログは「500だった」しか語らない。

原因は下流の機能サーバにある。

ここで止まるツールが、実務でいちばん多い。

手元で読めるログだけを見て「原因不明」と結論してしまう。

デモ ③ 別サーバのログをSSHで収集

抽出した TrackID をキーに、機能サーバのログを検索する。

```
ssh -i docker/sample-data/training_key -p 5101 trainee@localhost \  
"grep TrackID:ABC1234 /app/logs/service.log"
```

受講者はこのコマンドを覚え不要。

接続先・ポート・鍵のパス・ログの場所は AGENT_SSH_GUIDE.md に集約しており、
そのファイルをAIエージェントに読ませるだけで、エージェント側が接続を組み立てる。

運用上の意味

「SSHが分かる人しか調査できない」という**属人化の解消**。
知識をドキュメントに置き、実行をエージェントに任せる。

デモ ④ 1件のインシデントに束ねる

TrackID をキーに両ログを結合し、人が読める1件の記録にする。

— INCIDENT —

TrackID : ABC1234

発生時刻: 2026-07-25T09:00:00.089Z

概要 : 在庫切れ商品の詳細取得で TypeError (undefined.map)
アプリ層は500を返しているが、発生源は機能層

関連ログ:

[app.log 09:00:00.123] ERROR /api/robots/RBT-DOG-02 status=500

[service.log 09:00:00.089] ERROR /internal/inventory/RBT-DOG-02 status=500

時系列に注目 — 機能層の .089 が、アプリ層の .123 より**先**。

TrackIDで束ねた瞬間に「どちらが原因側か」が読める。時刻の目視では出てこない情報。

収集ツールを4段階に分けて考える

受講者が段階的に作れるよう、機能を4つに分解して教えている。

段階	やること	この環境での具体例	難所
① 検知	異常に気づく	<code>app.log</code> の <code>ERROR</code> 行	低
② 収集	関連ログを集める	SSHで <code>service.log</code> を検索	中(接続)
③ 関連付け	1件にまとめる	TrackIDで集約	高(設計)
④ 出力	人が読める形にする	一覧・レポート・通知	低

①②だけを先に動かし、通ってから③④を足す。

最初から全部を一度に指示すると、AIエージェントは的外れな実装に流れやすい。

AIエージェントに渡すべき前提

「ログ収集ツールを作って」だけでは通らない。**この環境固有の前提**を最初に渡す。

- **目的** — client/server 2つのログをTrackIDで紐づけ、1件のインシデントにする
- **ログの場所** — `client/logs/app.log` (ローカル) と server側(SSH、`AGENT_SSH_GUIDE.md` 参照)
- **相関キー** — `TrackID:` + 英数字7文字 / `TrackID:([A-Z0-9]{7})`
- **検知条件** — どの行を異常とみなすか(例: `ERROR` を含む行)
- **スコープ** — 今回作るのは①②まで(段階的に依頼する)
- **出力形式** — コンソール / ファイル / JSON

これは**プロンプトの型**であり、ログ収集に限らず流用できる。

研修の本当の成果物はツールではなく、**この型を書けるようになった担当者**。

Before / After

観点	Before(現状)	After(本運用)
ログの突き合わせ	時刻を目視、推測	TrackID完全一致、機械的
別サーバのログ取得	SSHを知る人が手作業	エージェントが自動収集
調べられる人	特定の担当者のみ	ガイドを読ませれば誰でも
記録の作成	手でコピー転記	自動で1件に整形
人がやること	探す作業 + 判断	判断に集中

本質的な変化は時間短縮ではなく、「**探す作業**」を人の仕事から外したこと。
探す作業は機械が得意で、判断は人が得意。分担を正した。

工数の試算

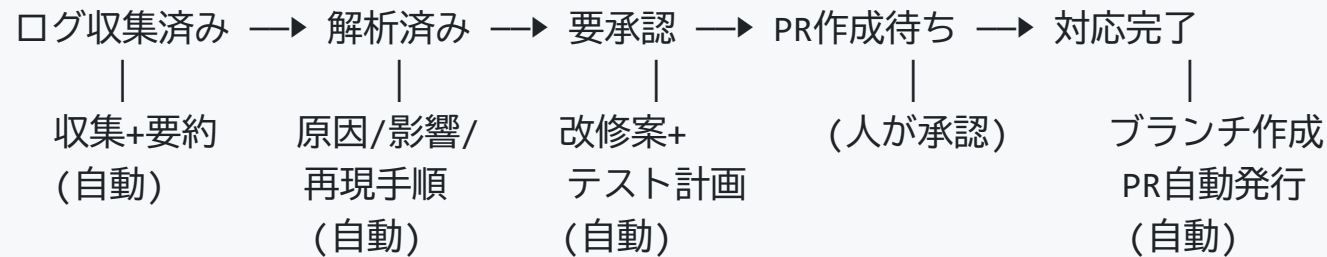
以下は実測値ではなく、検討用の仮定値です。導入判断には自組織の実績で置き換えてください。

作業	Before	After	前提
ログの所在特定	10分	0分	対象が定義済み
SSH接続・grep	15分	0分	エージェントが実行
突き合わせ	20分	0分	TrackID一致
内容確認・判断	10分	10分	人が担う(変えない)
起票・転記	10分	0分	自動整形
合計	約65分	約10分	1インシデントあたり

仮に月20件なら、月あたり 約18時間の削減に相当(同じ仮定の下での計算)。実運用では収集失敗時の切り分けなどが加わる。

展望 ①: 検知から改修PRまで繋ぐ

同一リポジトリの `projects/log_collector/auto-repair-demo` で**実証済み**の延長線。



各段を Claude Code のサブエージェントが担当:

`log-summarizer` → `incident-analyzer` → `repair-planner` → `pr-publisher`

状態は Excel 1行として管理され、**どの段で止まっているかが一目で分かる。**

運用の可視性を、新しいダッシュボードを作らずに確保している。

展望 ②: ガードレール — 人が止める場所を決める

自動化する範囲

- ログ収集
- 事象の要約
- 原因の一次解析
- 改修案とテスト計画の作成
- ブランチ作成とPR発行

人が必ず入る場所

手動ゲート① 起票直後

収集ログと概要を見て、調査を続けるか判断

手動ゲート② 改修案の承認

要承認 → PR作成待ち は人手のみ

main への直接反映はしない。 エージェントはブランチとPRまで。

マージは通常のレビュープロセスに乗る。AIが妙な改修案を出しても、PR発行前に人が止まる。

オンプレ環境での制約と対応

このネットワークのホワイトリストは `docker.io` と `npm` のみ。

場面	効くレイヤー	対応
① <code>docker build</code> 中のパッケージ取得	Dockerfile内のシェル環境変数	ビルド引数でプロキシを渡す。 イメージには焼き込まない
② <code>docker pull</code>	Dockerデーモンの設定	ホスト側で一度だけ設定 (スクリプト範囲外)
③ コンテナ実行時	コンテナの環境変数	compose の <code>environment:</code> で渡す

`apk add` は Alpine 独自ミラーを使うため、**プロキシを通しててもホワイトリスト外**。
→ 制約のない環境で**一度だけビルドして Docker Hub へ push**、オンプレでは `docker pull` のみ。

切替は `.env` の `USE_PROXY` 一つ。

導入ステップ

Step	内容	主体	完了条件
1	serverイメージをビルドし Docker Hub へ公開	環境構築担当	オンプレで <code>docker pull</code> が通る
2	研修環境を起動し、2つの不具合を再現	受講者	両ログに同一TrackIDを確認
3	AIエージェントで①検知②収集を実装	受講者	別サーバのログが取れる
4	③関連付け④出力を追加	受講者	1件のインシデントとして出力
5	自組織のログ形式へ相関キーを導入	開発チーム	本番ログにTrackID相当が入る
6	改修PR自動発行まで拡張	運用チーム	承認ゲート付きで運用開始

Step 5 が本番適用の分水嶺 — 相関キーが本番ログに入っていないければ、どんなツールも推測に戻る。**ログ設計が先、ツールは後。**

リスクと対策

リスク	影響	対策
AIが誤った原因・改修案を出す	誤った修正が入る	PR発行前に人の承認ゲート。 main へは直接反映しない
対象TrackIDのログが0件	起票内容が不完全	収集結果を記録に残し、 収集失敗が見えるようにする
ログ形式が変わる	抽出が壊れる	相関キーの形式を仕様として固定
研修環境の簡略化を本番に持ち込む	セキュリティ事故	鍵の平文同梱・SSH常時起動は学習用限定 と明記済み
属人化が別の形で再発	元に戻る	前提はドキュメントに集約 (AGENT_SSH_GUIDE.md の思想)

まとめ

1. 障害調査の時間は「判断」ではなく「**ログを探して突き合わせる作業**」に消えていた。
2. 全リクエストに **TrackID** を通せば、突き合わせは推測から**完全一致検索**になり、自動化できる。
3. ツールを配るのではなく、担当者がAIエージェントで**自分で組み立てられる状態**を作る。それが `onprem_log_training` 。
4. 改修PR自動発行までの延長線は実証済み。ただし**承認ゲートは人が持つ**。

次の判断事項

本番ログへの相関キー導入(Step 5)を、どの系から始めるか。

参考資料

projects/onprem_log_training/

ドキュメント	内容
README.md	起動手順・プロキシ・オンプレ運用
AGENT_SSH_GUIDE.md	AIエージェント向け接続情報
docs/01_environment_overview.md	環境の全体構成とTrackID
docs/02_building_log_collector_with_ai.md	プロンプト設計のポイント
docs/03_container_basics.md	コンテナとVMの違い
docs/infographics/	本資料のインフォグラフィック (一枚絵 + 詳細4枚)

projects/log_collector/auto-repair-demo/DEMO_FLOW.md — 自動改修フローの詳細